ETL Example—Load

Explore how to complete the ETL pipeline by loading cleaned data into PostgreSQL. Learn to create and use Bash scripts to manage the database environment and SQL scripts to create tables and import CSV data, ensuring a smooth transfer from processing to storage.

The last step in the ETL pipeline is to load the cleaned and processed data stored in `clean_data.csv` into a PostgreSQL database.

Connecting to the database

First, we create a Bash script called `load_data.sh` that will switch users from "root" to "postgres," start a Postgres database, and run an SQL script to load the data. The SQL script will be stored in a file called `load_data.sql`.

load_data.sh Shell

```shell
#!/bin/bash

# Setting the project location as an environment  variable
export WORKDIR='/usercode/ETL_Lottery'

# Setting Postgres bin and data directories as environment variables
export POSTGRES_BIN='/usr/lib/postgresql/16/bin'
export POSTGRES_DATA='/var/lib/postgresql/data'

# Create and start a new PostgreSQL database cluster.
# Connect to Postgres and run the file: load_data.sql
su - postgres -c "
```

Ask

Deploying a PostgreSQL database: load_data.sh

- **Lines 4–8**: We set the project location and Postgres bin directory as environment variables.
- **Lines 12–15:** We deploy a PostgreSQL database and run the SQL script stored in `load_data.sql`.
  - We switch users to “postgres.”
  - We run `initdb` to start a database cluster and create the necessary files and directories. We add the flag `–D` to specify the location of the data folder.
  - We use the utility `pg_ctl` to start the database cluster. We add the flag `–D` to specify the location of the data folder.
  - Lastly, we connect to the database and run the file `load_data.sql`.

Loading the data

The next step is to create the script for `load_data.sql`. Notice that `load_data.sh` is calling `load_data.sql` just after deploying the PostgreSQL database. `load_data.sql` will be in charge of creating a new table in the database and loading the clean data.

Files

load_da...

main.sh

load_data.sql

```
-- # Create table called: winning_numbers
drop table if exists winning_numbers;

create table winning_numbers(
date date not null,
first_num int not null,
second_num int not null,
third_num int not null,
fourth_num int not null,
fifth_num int not null,
sixth_num int not null,
multiplier int not null);
```

Ask

Creating a table and loading the data: load_data.sql

- **Line 4:** We create an empty table called `winning_numbers` with the above columns.
- **Line 15:** We load the file `clean_data.csv` to the new table `winning_numbers`.
  - `FROM` represents the location of the file.
  - `DELIMITER` represents the character that marks the end of a unit of data. In our case, it's commas.
  - `CSV HEADER` keeps the header row.
- **Line 18:** We query the first ten rows of `winning_numbers`.

The last thing we should do is update the file `main.sh` to include the `load_data.sh` file, and the ETL is done. The file `main.sh` now contains the ETL's entire extract, transform, and load steps.

We now have a tool that can download raw lottery data, process it according to specific business needs, and finally deliver clean data ready to be queried in a PostgreSQL database. Connect to the terminal widget below to run `main.sh`.

Terminal 1

Terminal

Loading...

Ask

- To connect to the server again and play around with the table `winning_numbers`, run these commands:
  - Switch to the user `postgres` by using the command `su postgres`.
  - Run the command `psql`.
- To exit the PostgreSQL server, use `\q`.

Mark As Completed

Ask